React, *props*, and the use of the curly brace. The curly brace is React's interpolation marker, similar to the way `<%= %>` is used in ERB. Inside curly braces, arbitrary JavaScript expressions are executed and passed through to the resulting element. In this case, we're taking the seat number and rendering each `Seat` component using its actual seat number.

Which brings us to *props*. Props—short for "properties," presumably—is React's term for the attributes of a component that are meant to be stable, meaning that the component cannot change its own props by itself. Props are passed into a component as a single argument when the component is created and are the only argument to a component. We could name the argument anything we want when defining the function; using `props` is a convention.

We're also using TypeScript to declare the type of our incoming props. In our case, we've declared `SeatProps` to say that the `props` object is expected to have only one key, `seatNumber`, and we expect that value to be a number. By declaring this interface, we allow TypeScript to do compile-time type checking to ensure that the necessary values are passed to our components. We can then use the `props` object, as we do here, by interpolating `props.seatNumber` into our JSX. I'll talk more about TypeScript interfaces in Chapter 6, *TypeScript*.

Destructuring



You'll also often see JavaScript destructuring syntax used to declare a component, in which case our declaration would be written as `export const Seat = ({ seatNumber })`. And then we could use that value in the component directly, with `<span className="button">{seatNumber}</span>`, since the incoming `props` object will have been deferenced directly to